gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843 Regression Testing (Part 2)

Dr. Francisco Gomes de Oliveira Neto (he/him)

Augusta Ada King, Countess of Lovelace

- English mathematician that lived between 1815 – 1852
- The **first** computer programmer

What we will learn today?

- How to use **diversity to prioritise** tests?
- Which algorithms can we identify diverse tests using a **distance matrix**?
- What is **APFD**? How to calculate it?
- How can we compare **different prioritisation criteria**?

© Francisco Oliveira Neto - 2018

Part 1:

Diversity-based Testing

© Francisco Oliveira Neto - 2022

When choosing tests…

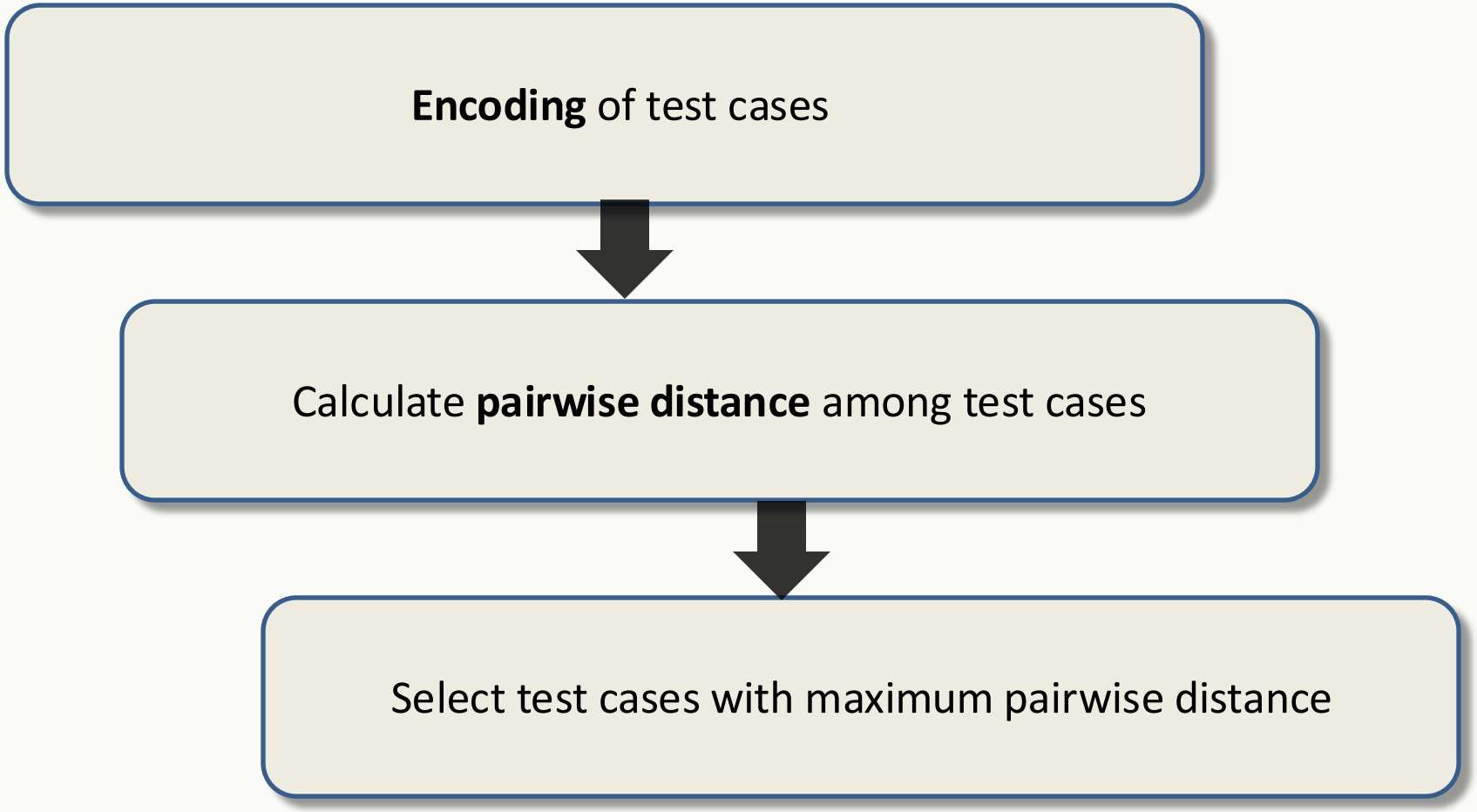
- We tend to be **diverse**…
- Specially if we **don't know** what to test.

Similar tests, tend to reveal the **same defect**.

Remember that **evidence** shows us that **faults are NOT uniformly** distributed across components

© Francisco Oliveira Neto - 2018

Diversity-based Testing (DBT)



Adapted from Hadi Hemmati, Andrea Arcuri, and Lionel Briand. 2013. Achieving scalable model-based testing through test case diversity. *ACM Trans. Softw. Eng. Methodol.* 22, 1, Article 6 (March 2013), 42 pages. DOI=http://dx.doi.org/10.1145/2430536.2430540

Encoding: Diversity is a number

TC1 - Add contact (create)
Select “Add contact”
Choose “Create New”
Fill in fields
Press ”Include contact”

TC2 – Add contact (import)
Select “Add contact”
Choose “Import”
Select an existing contact
Press ”Include contact”

TC3 – Remove a contact
Select “Remove contact”
Select one contact
Press “Remove Contact(s)”

© Francisco Oliveira Neto - 2018

Encoding: Distance / Similarity functions

$NCD_Z(x,y) = \frac{Z(xy) - \min\{Z(x), Z(y)\}}{\max\{Z(x), Z(y)\}}.$	**Normalized Compressed Distance**
$d(\vec{x},\vec{y}) = \sqrt{(\vec{x}-\vec{y})^T S^{-1} (\vec{x}-\vec{y})}.$	**Mahalanobis**
$\mathrm{NGD}(x,y) = \frac{\max\{\log f(x), \log f(y)\} - \log f(x,y)}{\log N - \min\{\log f(x), \log f(y)\}}$	**Normalized Google Distance**
$d_J(A,B) = 1 - J(A,B) = \frac{\lvert A \cup B\rvert - \lvert A \cap B\rvert}{\lvert A \cup B\rvert}.$	**Jaccard Distance**
$\mathrm{lev}_{a,b}(i,j) = \begin{cases} \max(i,j) \\ \min \begin{cases} \mathrm{lev}_{a,b}(i-1,j) + 1 \\ \mathrm{lev}_{a,b}(i,j-1) + 1 \\ \mathrm{lev}_{a,b}(i-1,j-1) + 1_{(a_i \neq b_j)} \end{cases} \end{cases}$	**Lenvenshtein Distance**

Diversity-based Testing - Example

TC1 - New contact (create)
Select “Add contact”
Choose “Create New”
Fill in fields
Press ”Include contact”

{ add, choose, contact, create, fields, fill, in, include, new, press, select }

TC2 – New contact (import)
Select “Add contact”
Choose “Import”
Select an existing contact
Press ”Include contact”

{ add, an, choose, contact, existing, import, include, press, select }

TC3 – Remove a contact
Select “Remove contact”
Select one contact
Press “Remove Contact(s)”

{ contact, contact(s), one, press, remove, select }

Diversity-based Testing - Example

TC1 - New contact (create)
Select “Add contact”
Choose “Create New”
Fill in fields
Press ”Include contact”

TC2 – New contact (import)
Select “Add contact”
Choose “Import”
Select an existing contact
Press ”Include contact”

TC3 – Remove one contact
Select “Remove contact”
Select one contact
Press “Remove Contact(s)”

TC4 – Remove several contacts
Select “Remove contact”
Select several contacts
Press “Remove Contact(s)”

TC5 – Update contact
Select “Update contact”
Choose one contact and field
Update the chosen field
Press “Confirm update”

Diversity-based Testing - Example

TC1 = {add, choose, contact, create, fields, fill, in, include, new, press, select}

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC5 = {and, choose, chosen, confirm, contact, field, one, press, select, the, update}

Diversity-based Testing - Example

	TC1	TC2	TC3	TC4	TC5
TC1					
TC2					
TC3					
TC4					
TC5					

We now need all **pairwise distance** / similarity values. Typically, we arrange them in a **matrix**. **Each test** case is a **row** and a **column** in the matrix.

Diversity-based Testing - Example

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

$$J(TC_i, TC_j) = \frac{|TC_i \cap TC_j|}{|TC_i \cup TC_j|}$$

Diversity-based Testing - Example

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC2 ∪ TC4 = {add, include, select, import, several, choose, an, contact(s), remove, existing, contact, press, contacts}

TC2 ∩ TC4 = {select, contact, press}

| TC2 | = 9 **| TC2 ∪ TC4 | = 13**

| TC4 | = 7 **| TC2 ∩ TC4 | = 3**

$$J(TC_2, TC_4) = \frac{|TC_3 \cap TC_4|}{|TC_3 \cup TC_4|} = \frac{3}{13} = 0.23$$

$$Distance(TC2, TC4) = 1 - 0.23 = 0.77$$

Note: Jaccard Index uses a **similarity index**, instead of a diversity index. So, **higher** values means **more similar** test cases. And for diversity, **we want low similarity** values.

Diversity-based Testing - Example

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC3 ∪ TC4 = {select, contact, one, several, contact(s), press, remove, contacts}

TC3 ∩ TC4 = {select, contact, contact(s), press, remove}

| TC3 | = 6 **| TC3 ∪ TC4 | = 8**

| TC4 | = 7 **| TC3 ∩ TC4 | = 5**

$$J(TC_3, TC_4) = \frac{|TC_3 \cap TC_4|}{|TC_3 \cup TC_4|} = \frac{5}{8} = 0.63$$

$$Distance(TC_3, TC_4) = 1 - 0.63 = 0.37$$

Diversity-based Testing - Example

	TC1	TC2	TC3	TC4	TC5
TC1					
TC2				0.77	
TC3				0.37	
TC4		0.77	0.37		
TC5					

$$D(TC_2, TC_4) = D(TC_4, TC_2) = 0.77$$

$$D(TC_3, TC_4) = D(TC_3, TC_4) = 0.37$$

Diversity-based Testing - Example

	TC1	TC2	TC3	TC4	TC5
TC1					
TC2				0.77	
TC3				0.37	
TC4		0.77	0.37		
TC5					

$$D(TC_2, TC_4) = D(TC_4, TC_2) = 0.77$$

$$D(TC_3, TC_4) = D(TC_3, TC_4) = 0.37$$

- Notice that 3 and 4 are more similar (i.e., "closer" to each other) if compared with TC2 and TC4.
- Using **distance values** we can find the **more "different"** tests from each other.

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

Exercise

- Calculate the rest of the similarity values.

TC1 = {add, choose, contact, create, fields, fill, in, include, new, press, select}

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC5 = {and, choose, chosen, confirm, contact, field, one, press, select, the, update}

Exercise

- Calculate the rest of the similarity values.

TC1 = {add, choose, contact, create, fields, fill, in, include, new, press, select}

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC5 = {and, choose, chosen, confirm, contact, field, one, press, select, the, update}

Jacc(TC1,TC2) = 6 / 14 = 0.43 → 1 – 0.43 = 0.57

Jacc(TC1,TC3) = 3 / 14 = 0.22 → 1 – 0.22 = 0.78

Jacc(TC1,TC4) = 3 / 15 = 0.20 → 1 – 0.20 = 0.80

Jacc(TC1,TC5) = 4 / 18 = 0.22 → 1 – 0.22 = 0.78

Jacc(TC2,TC3) = 3 / 12 = 0.25 → 1 – 0.25 = 0.75

Jacc(TC2,TC4) = 3 / 13 = 0.23 → 1 – 0.23 = 0.77

Jacc(TC2,TC5) = 4 / 16 = 0.25 → 1 – 0.25 = 0.75

Jacc(TC3,TC4) = 5 / 8 = 0.63 → 1 – 0.63 = 0.37

Jacc(TC3,TC5) = 4 / 13 = 0.31 → 1 – 0.31 = 0.69

Jacc(TC4,TC5) = 3 / 15 = 0.20 → 1 – 0.20 = 0.80

© Francisco Oliveira Neto - 2022

Diversity-based Testing - Example

	TC1	TC2	TC3	TC4	TC5
TC1	---	0.57	0.78	0.80	0.78
TC2	0.57	---	0.75	0.77	0.75
TC3	0.78	0.75	---	0.37	0.69
TC4	0.80	0.77	0.37	---	0.80
TC5	0.78	0.75	0.69	0.80	---

This is a similarity / distance matrix.

The **matrix is symetrical**, because:

D(a,b) == D(b,a)

The **diagonal is ignored**, because the distance of an object to itself is always zero (or similarity is always one).

Diversity-based Testing - Example

- We can use the matrix for all sorts of operations.
- We can find diverse subsets: Pick the **highest values one at a time**.
- We can see whether we need to create more different tests.

	TC1	TC2	TC3	TC4	TC5
TC1	---	0.57	0.78	0.80	0.78
TC2		---	0.75	0.77	0.75
TC3			---	0.37	0.69
TC4				---	0.80
TC5					---

Diversity-based Prioritisation: MaxMin

- **Idea:** 1) Find the large distance. 2) Update the matrix to keep the min distances.
- Find the **highest distance**: save that pair in a **list**
- current_vec = Update to the **minimal distances** to each of these pairs

- The loop is to: Until you **prioritised all tests** (or reach a **limit test suite size**):
  - Find the TC maximum distance value to the current_vec.
  - Add that TC to the list of prioritised
  - Update the distances to keep the minimal ones

© Francisco Oliveira Neto - 2022

Diversity-based Testing - Example

	TC1	TC2	TC3	TC4	TC5
TC1	---	0.57	0.78	0.80	0.78
TC2	0.57	---	0.75	0.77	0.75
TC3	0.78	0.75	---	0.37	0.69
TC4	0.80	0.77	0.37	---	0.80
TC5	0.78	0.75	0.69	0.80	---

MAX-MIN – Description of how to prioritise:

1: Find the **highest distance value** (choose how to break ties):

- Both (T1,T4) and (T4,T5) have high distance (0.8). Lets choose (randomly T1,T4).

2: We add **the first pair** to our Prioritised Set P = [T4, T1]

Then we iterate over the rest of tests (individually) picking one at a time.

3: Create a temp vector with the minimal distances between each test from the chose: min vector between columns TC1, TC4.

- Temp = [---, 0.57, 0.37, ---, **0.78**].
- We pick the max then which is 0.78, so T5 is added to the P.
- P = [T4, T1, T5]

4: Update the Temp vector with the min distance between the **remaining columns**:

- Temp = [---, 0.57, 0.37, ---, ---] ex: 0.57 is min(0.57,0.77,0.75)

5: Repeat steps 3 and 4 above until you run out of tests.

- Temp = [---, **0.57**, 0.37, ---, ---] max distance in Temp is T2.
- P = [T4, T1, T5, **T2**]
- Temp = [---, ---**, 0.37**, ---, ---]. 0.37 is min(0.78,0.75,0.37,0.69)
- P = [T4, T1, T5, T2, T3]. **No more tests left, we stop.**

Diversity-based Testing – Example 2

	TC1	TC2	TC3	TC4	TC5
TC1	---	0.57	0.78	0.80	0.78
TC2	0.57	---	0.75	0.77	0.75
TC3	0.78	0.75	---	0.37	0.69
TC4	0.80	0.77	0.37	---	0.80
TC5	0.78	0.75	0.69	0.80	---

MAX-SUM – Description of how to prioritise:

1: Find the **highest distance value** (choose how to break ties):

- Both (T1,T4) and (T4,T5) have high distance (0.8). Lets choose (randomly T1,T4).

2: We add **the first pair** to our Prioritised Set P = [T4, T1]

Then we iterate over the rest of tests (individually) picking one at a time.

3: Create a temp vector with the sum of distances the chosen columns so far: TC1, TC4.

- Temp = [---, 1.34, 1.15, ---, **1.58**]. 1.34 = 0.57(tc2,tc1) + 0.77(tc2,tc4)
- We pick the max then which is 1.58, so T5 is added to the P.
- P = [T4, T1, T5]

4: Update the Temp vector with the sum distance between the **latest column**:

- Temp = [---, 2.09, 1.84, ---, ---] ex: 2.09 is 1.34 + 0.75 (tc2, tc5)

5: Repeat steps 3 and 4 above until you run out of tests.

- Temp = [---, **2.09**, 1.84, ---, ---] max distance in Temp is T2.
- P = [T4, T1, T5, **T2**]
- Temp = [---, ---, **2.59**, ---, ---]. 2.59 is 1.84 + 0.75 (tc3,tc2)
- P = [T4, T1, T5, T2, T3]. **No more tests left, we stop.**

Diversity-based Testing - Example

Prioritised test suite = [T4, T1, T5, T2, T3].

TC4 – Remove several contacts
Select “Remove contact”
Select several contacts
Press “Remove Contact(s)”

TC1 - New contact (create)
Select “Add contact”
Choose “Create New”
Fill in fields
Press ”Include contact”

TC5 – Update contact
Select “Update contact”
Choose one contact and field
Update the chosen field
Press “Confirm update”

TC2 – New contact (import)
Select “Add contact”
Choose “Import”
Select an existing contact
Press ”Include contact”

TC3 – Remove one contact
Select “Remove contact”
Select one contact
Press “Remove Contact(s)”

Part 1: Take-aways

- Diversity is a **powerful measure** to indicate redundancy or repetition between tests.
- Diversity can be measured in **any artefact** (text, numbers, images)
- Diversity relies on a **distance function**. We calculate distance between pairs of tests. It **can be inneficient**.
- We can **prioritise tests** based on diversity (or similarity).

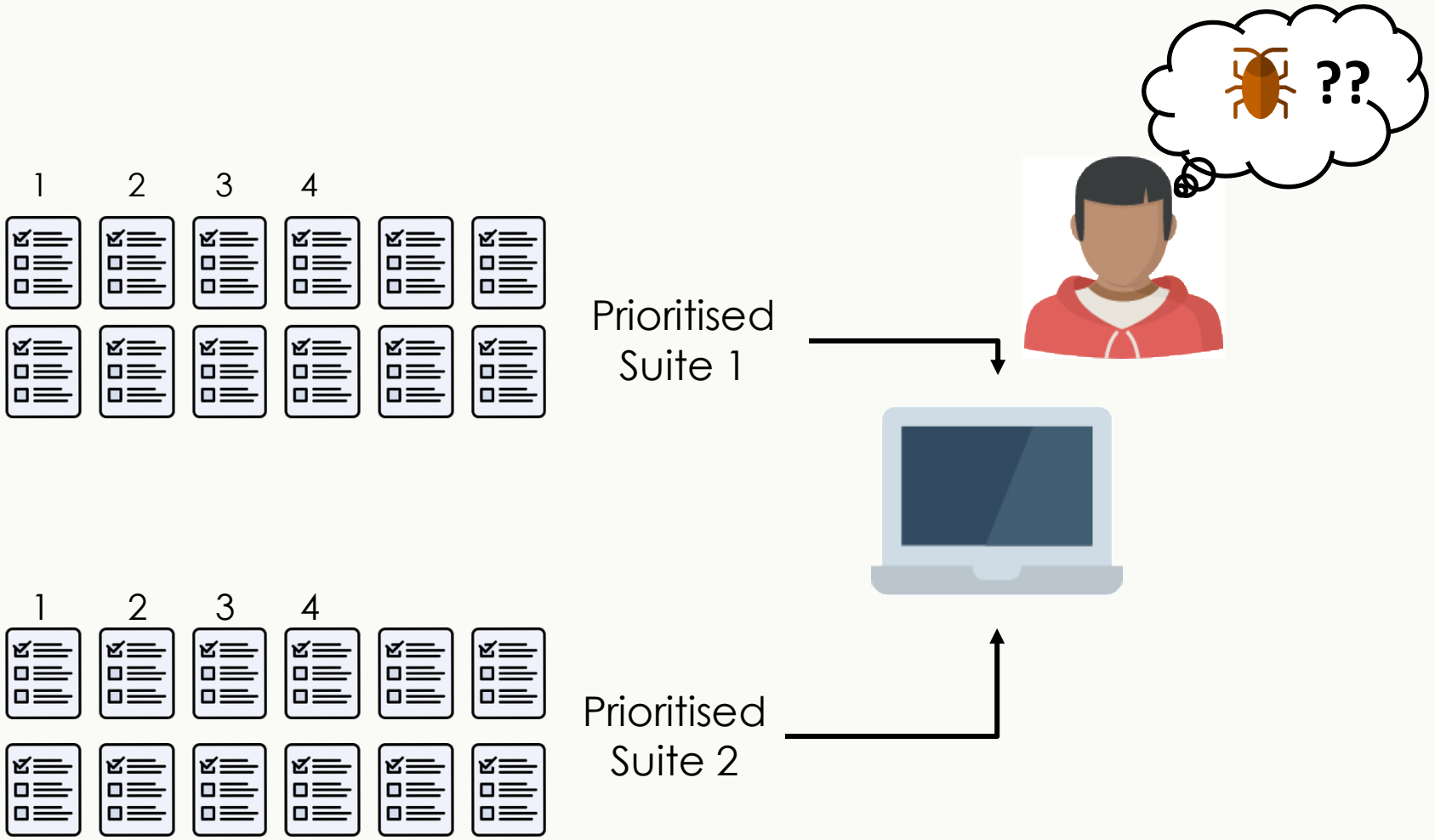
© Francisco Oliveira Neto - 2022

Part 2:

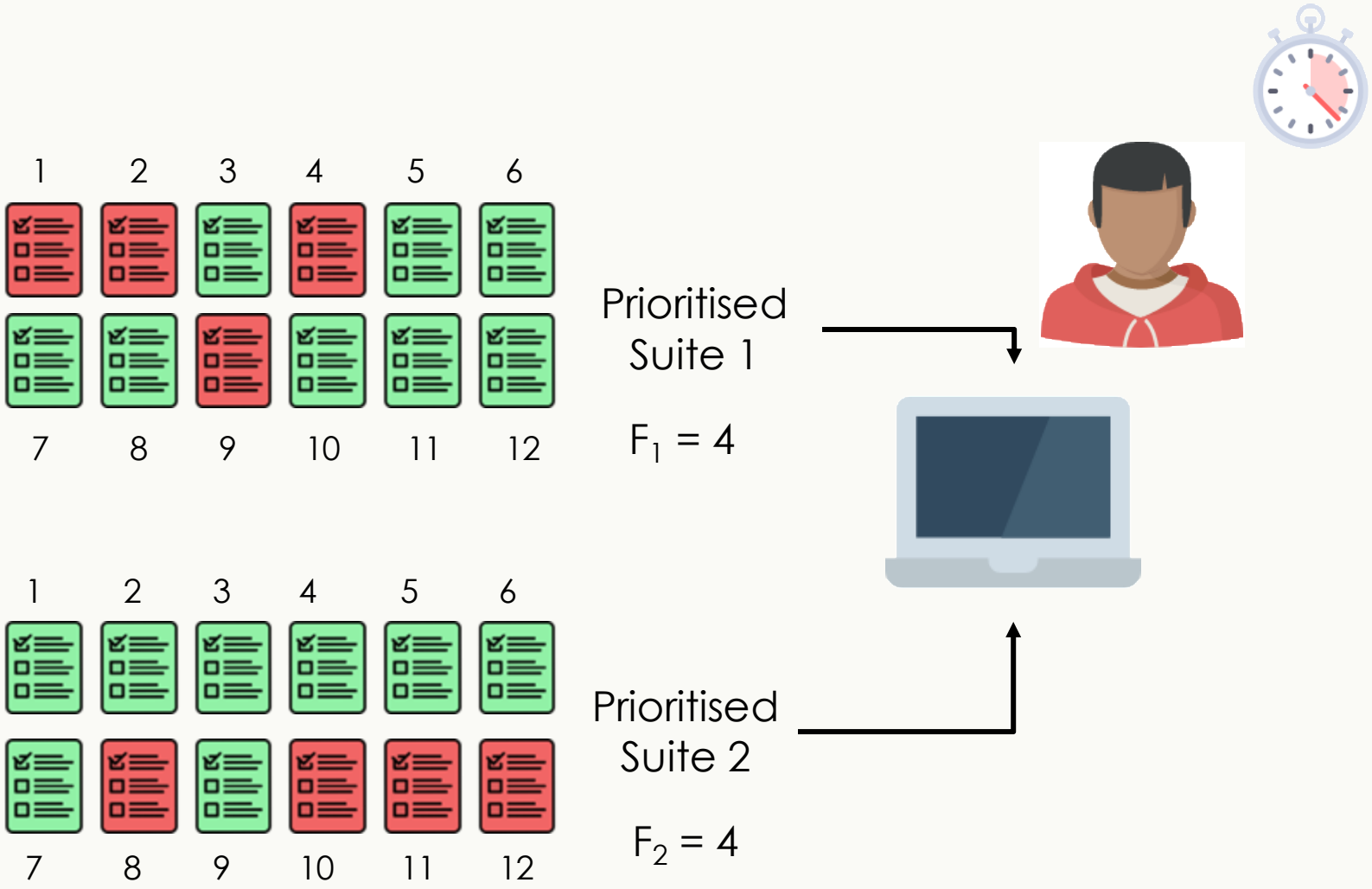
Evaluating Test Optimisation

© Francisco Oliveira Neto - 2022

Evaluating prioritisation techniques



Evaluating prioritisation techniques



What is APFD?

- **APFD (Average Percentage of Faults Detected)** is a metric that quantifies **how quickly** a test prioritization technique uncovers faults.

- It measures the **area under the fault-detection curve**: the earlier a prioritization places tests that detect faults, the higher the APFD value (range: 0–1 or 0–100%).

© Francisco Oliveira Neto - 2022

What is APFD?

$$\mathrm{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$

0 ⟷ 100

Higher numbers

Faster detection rates

APFD

Fault = Defect

$$\text{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$

TF_i = is the **order** where we revealed **i-th fault**

Each “i” is a fault

n = total number of **test cases**

m = total number of **faults**

APFD

$$\text{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$

	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
TC1				X		X				
TC2		X								
TC3		X							X	
TC4		X	X						X	
TC5			X			X				X
TC6										
TC7	X									
TC8				X				X		
TC9										
TC10					X	X	X	X		

TP1 = [TC10, TC2, TC5, TC1, TC8, TC9, TC3, TC7, TC4, TC6]

TP2 = [TC6 , TC3, TC9, TC1, TC4, TC2, TC5, TC8, TC7, TC10]

APFD

$$\text{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$

	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
TC1				X		X				
TC2		X								
TC3		X							X	
TC4		X	X						X	
TC5			X			X				X
TC6										
TC7	X									
TC8				X				X		
TC9										
TC10					X	X	X	X		

TP1 = [TC10, TC2, TC5, TC1, TC8, TC9, TC3, TC7, TC4, TC6]

Positions: 1 2 3 4 5 6 7 8 9 10

F1 = 8 F4 = 4 F7 = 1 F10 = 3

F2 = 2 F5 = 1 F8 = 1

F3 = 3 F6 = 1 F9 = 7

F1 + ... + F10 = 31

APFD

$$\text{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$

	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
TC1				X		X				
TC2		X								
TC3		X							X	
TC4		X	X						X	
TC5			X			X				X
TC6										
TC7	X									
TC8				X				X		
TC9										
TC10					X	X	X	X		

$$APFD_{TP2} = 1 - \frac{\mathbf{61}}{10\ x\ 10} + \frac{1}{2\ x\ 10}$$

$$= 1 - 0.61 + 0.05$$

$$= 0.44 = \mathbf{44}\%$$

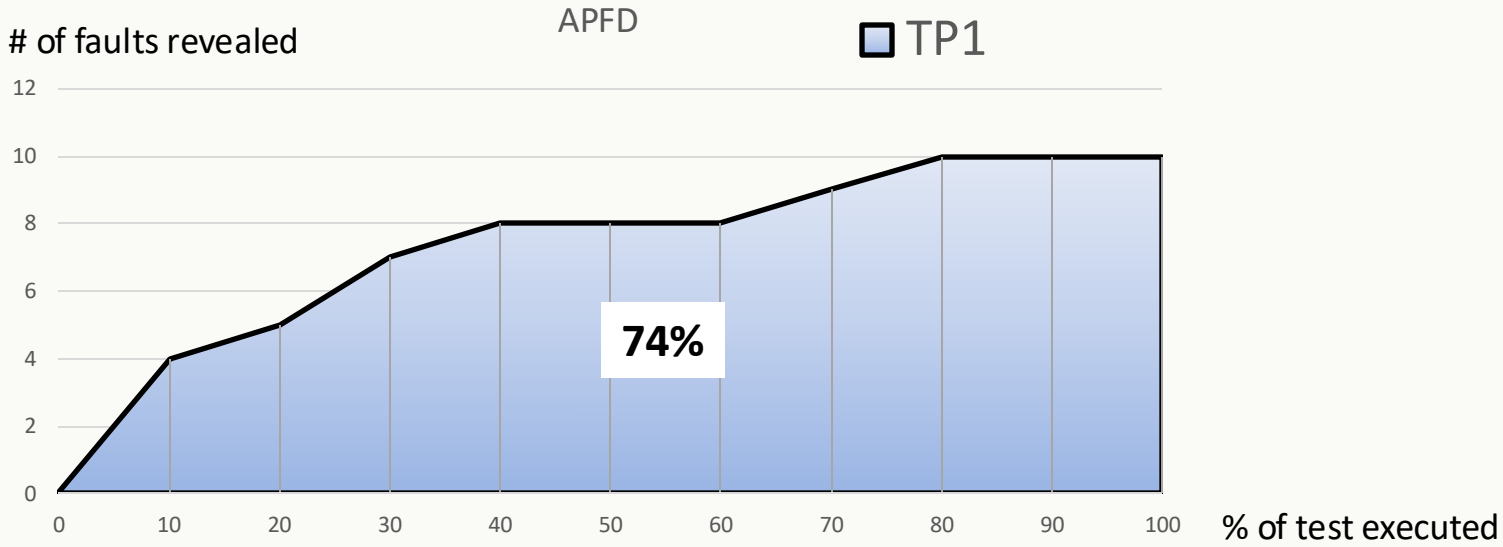
TP1 = [TC10, TC2, TC5, TC1, TC8, TC9, TC3, TC7, TC4, TC6]

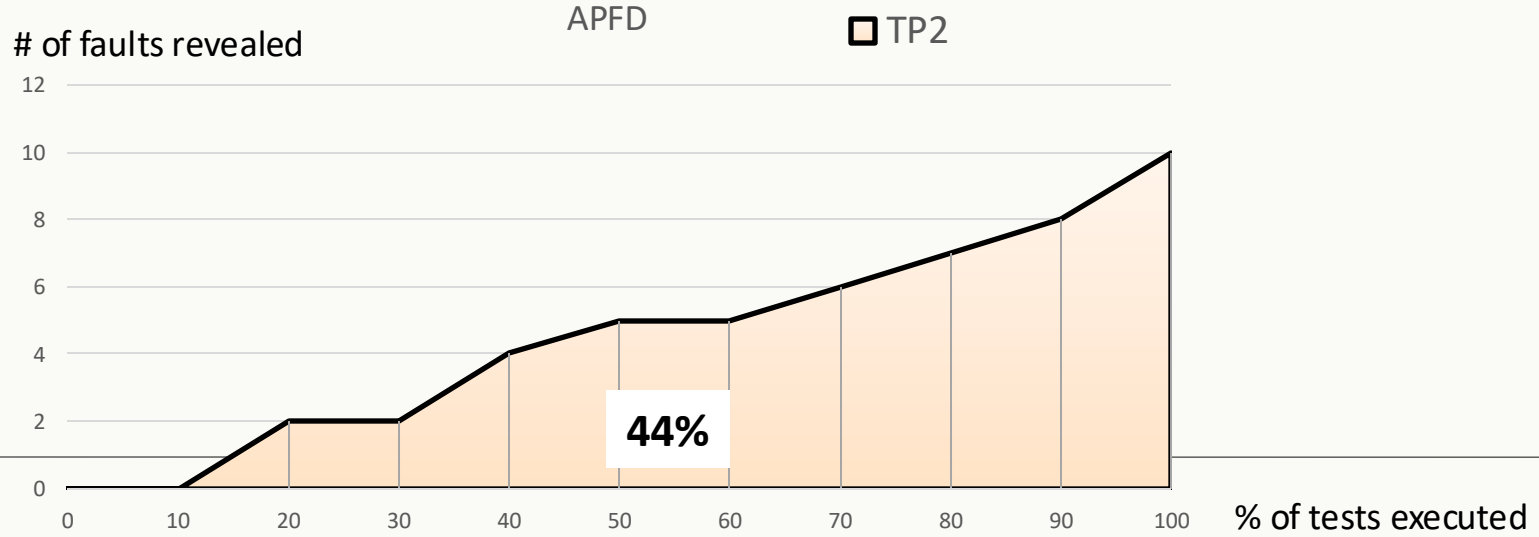
$$APFD_{TP1} = 1 - \frac{\mathbf{31}}{10\ x\ 10} + \frac{1}{2\ x\ 10} = 1 - 0.31 + 0.05 = 0.74 = \mathbf{74}\%$$

TP2 = [TC6 , TC3, TC9, TC1, TC4, TC2, TC5, TC8, TC7, TC10]

APFD

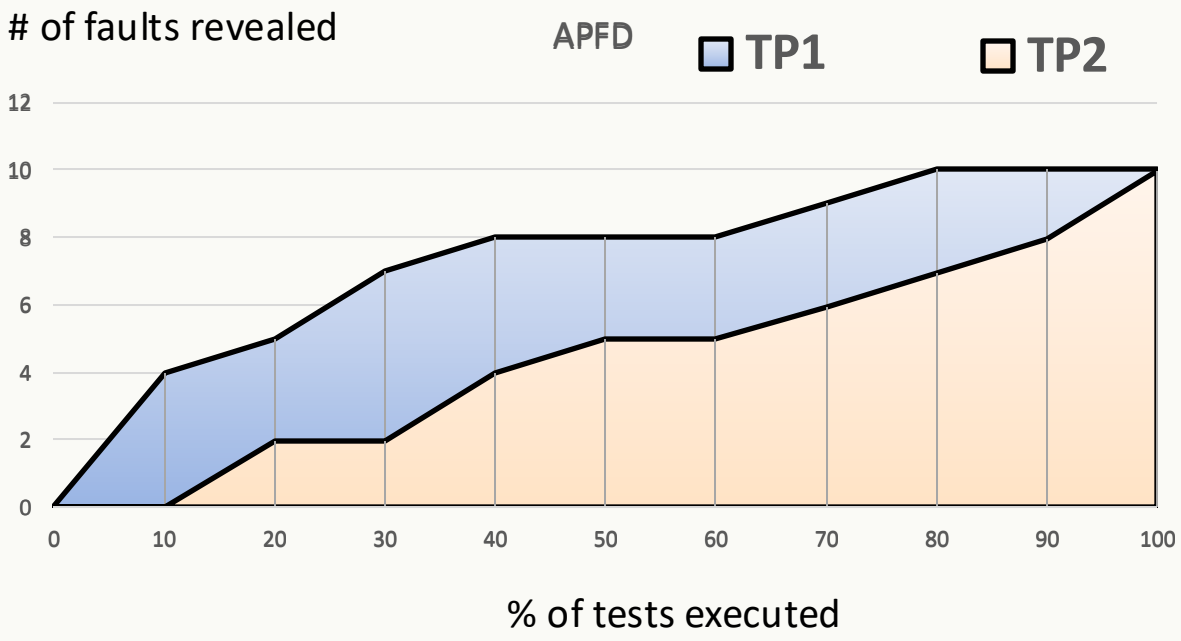
$$\text{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$





APFD

$$\text{APFD} = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{nm} + \frac{1}{2n}$$



Exercise - APFD

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
F1	X			X						
F2			X				X			
F3	X						X			
F4				X				X		
F5						X				
F6				X						X

- Consider the three prioritised test suites below.

- Use APFD to find the better test suite:
  - `SUITE-A: [T1, T3, T6, T8, T7, T2, T5, T9, T4, T10]`
  - `SUITE-B: [T3, T2, T1, T5, T6, T8, T10, T4, T7, T9]`
  - `SUITE-C: [T2, T5, T9, T3, T7, T1, T8, T6, T10, T4]`

© Francisco Oliveira Neto - 2022

Part 2: Take-aways

- APFD tells you how **early faults are detected** — not just how many.

- A high APFD ≠ the prioritization is perfect; it only reflects **fault positions**.
  - It does not account for execution time, severity of faults, etc.

- APFD is great for **comparing prioritization** techniques, but limited in real datasets.
  - We cannot know the faults in advance. So, history / statistics are an estimation.

© Francisco Oliveira Neto - 2022